TSS / PROIECT DE CURS 14

Testarea bazată pe proprietăți cu fundamente formale

Foundational Property-Based Testing în Haskell și Coq/QuickChick

Prezentatori: Natalia Moroz (Teorie) / Victor Mura (Demo)

Durată: 20 de minute | 10 minute per membru | Echipa: 50% - 50%

PREZENTAREA PE SCURT

01 **TEORIE & PROBLEMĂ**
Natalia: Ce este PBT, mecanismul QuickCheck și limitele cutiei negre

02 **FUNDAMENTE FORMALE**
Natalia: semGen, semChecker și cele 3 teoreme formale din articol

03 **DEMO HASKELL LIVE**
Victor: Arhitectura FGen a și cele 5 experimente comparative în terminal

Agenda Prezentării

PARTEA 1: TEORIE & PROBLEMĂ
Natalia Moroz (~6 min)

- Ce este Property-Based Testing (PBT)
- Algoritmul QuickCheck în 4 pași
- Vulnerabilitățile cutiei negre
- Soluția: Teoria Foundational PBT (semGen)

PARTEA 2: DEMO ÎN HASKELL
Victor Mura (~10 min)

- Arhitectura motorului formal (FGen a)
- Scenariul de tranzacții financiare
- Cele 5 experimente comparative rulate live
- Izolarea defectelor vs. certificare formală

PARTEA 3: CONTEXT & CONCLUZII
Natalia Moroz (~4 min)

- Studiul de caz din articol (Arbori Roșu-Negru)
- Ce garantează metoda și limitele practice
- Tabelul sinoptic comparativ
- Trei idei de reținut & Sesiune Q&A

Organizarea celor 20 de minute

Ce este Property-Based Testing și de ce îl folosim?

TESTARE PE EXEMPLE (CLASICĂ)

• Verifică puncte izolate scrise manual de dezvoltator.

• Exemplu: assert(adunare(2, 3) == 5)

• Problema: Subiectivitatea umană ratează cazurile limită (edge-cases: valori negative, zero, numere uriașe, liste goale).

• Rezultat: Testele trec, dar aplicația crapă în producție pe date neprevăzute.

PROPERTY-BASED TESTING (PBT)

• Schimbă complet paradigma: definește reguli universale (proprietăți) ce trebuie să rămână mereu adevărate.

• Exemplu: adunare(a, b) == adunare(b, a) (comutativitate pentru orice valori).

• Mecanism: Generatorul inundă automat funcția cu mii de valori diverse și cazuri extreme.

• Shrinking: La eșec, reduce automat contraexemplul la cea mai simplă formă care produce bug-ul.

DE CE ÎL FOLOSIM? Trecem de la «funcționează pentru exemplul meu ales manual» la «funcționează pentru orice intrare validă posibilă», identificând defecte ascunse înainte de lansare.

Cum funcționează PBT? Cei 4 pași algoritmici

01

PROPRIETATE
O regulă executabilă generală

Definim o aserțiune universală ce trebuie să rămână adevărată pentru orice date.
Ex: decode (encode tx) == Just tx

02

GENERATOR
Generare pseudo-aleatorie

Un algoritm produce automat sute/mii de instanțe aleatorii, acoperind cazuri limită precum valori nule sau extreme.

03

RULARE
Evaluare automată

Execută proprietatea pe fiecare set de date produs de generator și înregistrează primul contraexemplu care pică.

04

REDUCERE
Shrinking automat

Dacă testul pică pe o intrare masivă, simplifică recursiv datele până găsește cel mai mic exemplu minimal de eșec.

Exemple clasice de proprietăți: Totalul coșului ≥ 0; decode (encode x) == Just x; Inversarea unei liste de două ori returnează lista originală (reverse (reverse l) == l).

Limitele PBT Clasic: Problema cutiei negre

01

GENERATOR INCOMPLET
Trecere Falsă (False Confidence)

Ce se întâmplă: Generatorul produce doar tranzacții în RON, ignorând EUR și USD.
Efectul în PBT clasic: Testul trece 100 de teste cu succes! Dezvoltatorul are o falsă siguranță, deși codul este defect pe EUR și USD.
Cauza: Generatorul are un unghi mort pe 66% din domeniu.

02

GENERATOR NEVALID
Eșec Fals (Spurious Failure)

Ce se întâmplă: Generatorul produce date ilegale (ex: ID negativ sau sume negative interzise de domeniu).
Efectul în PBT clasic: Testul pică și dă vina pe codul bun al aplicației!
Cauza: Generatorul încalcă precondiția de business; dezvoltatorul pierde timp căutând bug-uri inexistente.

03

VERIFICATOR ERONAT
Discrepanță de Specificație

Ce se întâmplă: Verificatorul executabil omite verificarea unui câmp (verifică doar suma, nu și moneda).
Efectul în PBT clasic: Codul cu bug trece neobservat pentru că verificatorul nu exprimă fidel specificația.
Cauza: Testul executabil diferă de contractul abstract.

Soluția: Foundational Property-Based Testing

INOVAȚIA DIN ARTICOLUL ITP 2015 (Paraskevopoulou, Hriţcu, Pierce et al.):
În loc să ghicim ce face generatorul, îi atașăm o semantică denotațională exactă: semGen(G) = mulțimea valorilor pe care generatorul le poate produce cu probabilitate nenulă. Astfel, verificăm formal codul de testare înainte de a rula vreun test!

TEOREMA 1
CORECTITUDINE (SOUNDNESS)
semGen(G) ⊆ { x | Φ(x) }
Garantează că generatorul nu produce gunoi: fiecare intrare generată respectă precondiția de domeniu. Niciun test nu va pica din cauza unui generator prost.

TEOREMA 2
COMPLETITUDINE (ADEQUACY)
Domain ⊆ semGen(G)
Garantează că generatorul nu are unghiuri moarte: 100% din cazurile de interes pot fi generate. Niciun bug nu poate scăpa ascuns într-o partiție negenerată.

TEOREMA 3
FIDELITATE (FIDELITY)
semChecker(c) ⟺ Spec
Garantează că verificatorul executabil este echivalent logic cu specificația abstractă. Testul acceptă exact ceea ce specificația declară valid.

Arhitectura Demo-ului Haskell

MOTORUL FORMAL (FoundationalPBT.hs)

• data FGen a = FGen { sampleOne :: Seed -> (a, Seed), rawSupport :: [a] }

• semGen :: Ord a => FGen a -> Set a — calculează suportul matematic exact.

• Monadă completă (Functor, Applicative, Monad): combinatorii compun atât eșantionarea, cât și semantica possibilistică.

• verifySoundness: verifică incluziunea în precondiție.

• verifyCompleteness: verifică acoperirea domeniului și calculează procentul exact de acoperire.

• verifyFidelity: găsește orice discrepanță între verificator și specificație.

DOMENIUL DE BUSINESS (Main.hs)

• data Transaction = Transaction { txId :: Int, amount :: Int, txCurrency :: Currency }

• encode tx: serializează în formatul id|cents|currency

• encodeBug tx: bug deliberat — ignoră moneda primită și scrie mereu RON!

• decode str: parsează stringul și verifică txId > 0 && amount >= 0

• specRoundTrip: decode (encode tx) == Just tx

• checkAmountOnly: verificator neglijent ce compară doar suma în bani.

• Trei generatoare: genComplete (complet & valid), genRonOnly (incomplet), genUnsound (ID negativ).

Demonstrație Live: Cele 5 Experimente în Terminal

EXP 1: GENERATOR INCOMPLET
PBT Convențional: PBT: Trecere falsă (100 teste trecute!) | Foundational PBT: FPBT: REJECTED (33% acoperire, lipsesc EUR & USD)

EXP 2: GENERATOR UNSOUND
PBT Convențional: PBT: Eșec fals (învinovățește codul corect) | Foundational PBT: FPBT: REJECTED (Izolează bug-ul în generator)

EXP 3: VERIFICATOR ERONAT
PBT Convențional: PBT: Trecere falsă (omite verificarea monedei) | Foundational PBT: FPBT: REJECTED (Găsește 18 deviații de la Spec)

EXP 4: TESTARE PE COD CU BUG
PBT Convențional: Eșuează probabilistic în PBT clasic | Foundational PBT: FPBT: Găsește contraexemplul minimal cert: Transaction 1 0 EUR

EXP 5: TESTARE PE COD CORECT
PBT Convențional: 100 teste trecute fără certitudine formală | Foundational PBT: FPBT: CERTIFICAT FORMAL EMIS (100% acoperire a claselor)

Comandă rulare terminal: cd demo && cabal run (sau ghc Main.hs FoundationalPBT.hs -o demo && ./demo)

Studiu de caz din articol: Inserarea în arbori roșu-negru

INVARIANTE DE ECHILIBRARE

01: Rădăcina și frunzele sunt negre.

02: Un nod roșu are doar copii negri.

03: Fiecare cale până la frunze conține același număr de noduri negre (înălțime neagră identică).

Problema: Un generator aleator generic produce extrem de rar arbori care respectă simultan toți acești invarianți structurali complecși.

REZULTATELE EXPERIMENTALE DIN ARTICOL (ITP 2015)

GENERATOR GENERIC (QuickCheck clasic cu filtru):

- Doar 2.415 teste trecute.
- Peste 20.000 de arbori invalizi au fost aruncați la gunoi!
- Testarea a renunțat din cauza ratei uriașe de respingere.
- Dimensiunea medie a arborilor testați: doar 1,1 noduri (complet irelevant pentru a prinde bug-uri reale).

GENERATOR FOUNDATIONAL (QuickChick certificat):

- 10.000 de teste trecute cu succes fără efort.
- 0 arbori aruncați (generare validă prin construcție demonstrată matematic).
- Dimensiunea medie a arborilor testați: 940,7 noduri (arbori giganți reali).

Ce susține fiecare abordare și care sunt limitele?

CE DEMONSTREAZĂ FOUNDATIONAL PBT

- Generatorul produce strict valori valide din precondiție (Soundness).
- Generatorul acoperă întreg domeniul de testare specificat (Completeness).
- Verificatorul executabil exprimă fidel specificația declarativă (Fidelity).
- Elimină complet defectele introduse de harness-ul de testare.

CE RĂMÂNE DE VERIFICAT (LIMITĂRI)

- Specificația scrisă trebuie să reflecte nevoile reale ale utilizatorului.
- Testarea rămâne o execuție finită: eșantionarea pe un spațiu infinit poate rata bug-uri extrem de rare.
- Nu înlocuiește demonstrația formală totală a programului, ci oferă o testare incontestabilă din punct de vedere al metodei.

TABEL SINOPTIC COMPARATIV

- PBT Convențional (QuickCheck, 2000): Presupune că proprietatea, generatorul și verificatorul sunt corecte. Oferă încredere pur statistică pe eșantionul rulat.
- Foundational PBT (QuickChick, 2015): Modelează matematic semGen și semChecker, demonstrând corectitudinea codului de testare înainte de execuție.

Concluzii: Trei idei de reținut

01

PROPRIETĂȚI
Comportament, nu exemple

PBT descrie comportamentul sistemului prin contracte universale (izomorfism, legi algebrice, reguli de business) în loc de exemple izolate hardcodate.

02

GENERATOARE
Calitatea decide încrederea

Un test trecut depinde critic de domeniul explorat. Un generator slab ascunde defecte în producție dând o falsă impresie de securitate.

03

FUNDAMENTE
Verificarea codului de testare

Foundational PBT elimină cutia neagră: verifică formal generatorul și verificatorul, transformând testarea într-un proces riguros și certificat.

Echipa de proiect: Natalia Moroz (50%) & Victor Mura (50%) — Vă mulțumim pentru atenție!

Referințe: [1] Z. Paraskevopoulou et al., 'Foundational Property-Based Testing', ITP 2015. [2] K. Claessen, J. Hughes, 'QuickCheck', ICFP 2000.